



Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Edison Leonardo Coronel Romero
Asignatura	Desarrollo Basado en Plataformas
Ciclo	5 A
Unidad	1
Título del producto	Desarrollo del Backend, Arquitectura Base y Modelo C4 del Sistema Multiplataforma
Nombre del Docente	Edison Leonardo Coronel Romero
Fecha	Jueves 06 de noviembre
Horario	07h30 – 10h30
Lugar	Laboratorio Virtual EVA Aula
Tiempo planificado en el Sílabo	3 horas
Integrantes	Andy Apolo, Justin Guitiérrez, Brandon Medina

Título del producto:

Desarrollo del Backend, Arquitectura Base y Modelo C4 del Sistema Multiplataforma

Propósito de la evaluación:

Evaluar la capacidad del estudiante para diseñar, implementar y documentar el backend del sistema multiplataforma, aplicando buenas prácticas de arquitectura, control de versiones, seguridad web y documentación profesional. Esta entrega constituye el primer hito del proyecto integrador, donde se define la arquitectura técnica base mediante el modelo C4 (niveles Context, Container y Component), que servirá de guía para las siguientes unidades.

Producto esperado:

Un **repositorio remoto (GitHub o GitLab)** correctamente estructurado y documentado que contenga:

1. **Backend funcional**, con:
 - Endpoints REST implementados (usuarios, autenticación y entidades base).

```
# general
path('', views.home, name='home'),
path('login/', views.log_in, name='login'),
path('register/', views.register, name='register'),
path('profile/', views.profile, name='profile'),
path('logout/', views.log_out, name='logout'),

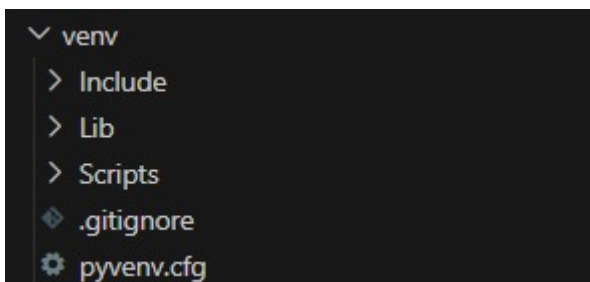
# notification
path('notification/change/<int:notification_id>/', views.mark_notification_as_read, name='notification'),

# users
path('user/state/udate/<int:user_id>/', views.change_state_user, name='change-user-state'),
path('user/desactivate/', views.desactivate_account_notification, name='desactivate-account'),
```

Los fragmentos de código definen todos los Endpoints REST principales de la aplicación. Esto incluye rutas para la Autenticación (/login/, /register/), la gestión de Usuarios (/profile/) y un endpoint específico para Notificaciones (/notification/change/<int:id>) que me permite manejar acciones como marcar una notificación como leída.

- Configuración del entorno, dependencias y estructura modular del proyecto.

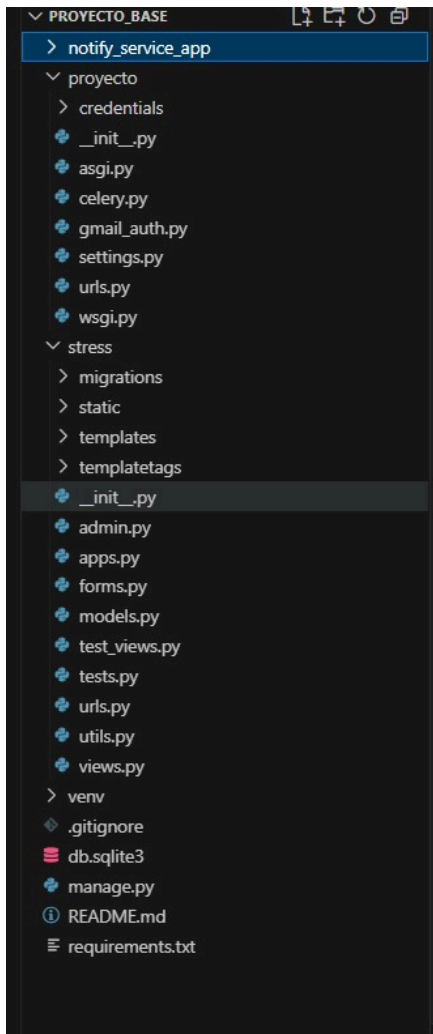
CONFIGURACIÓN DEL ENTORNO:



```
.ps1
(venv) PS C:\Users\apolo\OneDrive\Escritorio\Stress_Track\proyecto_base> .\venv\Scripts\activate
(venv) PS C:\Users\apolo\OneDrive\Escritorio\Stress_Track\proyecto_base> pip install -r requirements.txt
```

Creamos un entorno virtual (venv) para aislar las librerías de mi proyecto. Luego, lo activé con `.\venv\Scripts\activate` para que la terminal sepa que estoy trabajando en él. Finalmente, usé el comando `pip install -r requirements.txt` para instalar todas las dependencias específicas que mi código necesita.

ESTRUCTURA MODULAR DEL PROYECTO:



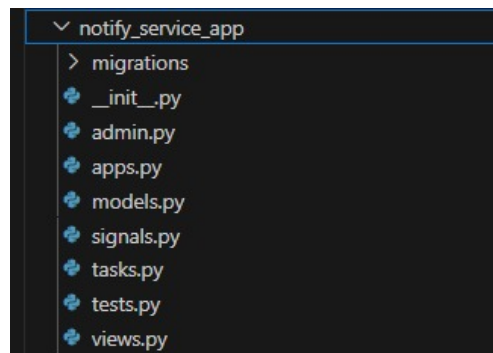
El proyecto tiene una estructura modular bien definida para mantener la organización y escalabilidad. El núcleo principal se divide en dos grandes módulos. El módulo stress contiene la lógica principal de la aplicación, como los modelos (models.py) y las vistas (views.py). El módulo notify_service_app está completamente independiente, funcionando como un microservicio dedicado exclusivamente a la gestión de notificaciones.

DEPENDENCIAS:

```
# requirements.txt
1  amqp==5.3.1
2  asgiref==3.10.0
3  billiard==4.2.2
4  cachetools==6.2.1
5  celery==5.5.3
6  certifi==2025.10.5
7  cffi==2.0.0
8  charset-normalizer==3.4.4
9  click==8.3.0
10 click-didyoumean==0.3.1
11 click-plugins==1.1.1.2
12 click-repl==0.3.0
13 colorama==0.4.6
14 Django==5.2.7
15 event==25.9.1
16 google-api-core==2.28.1
17 google-api-python-client==2.187.0
18 google-auth==2.41.1
19 google-auth-httplib2==0.2.1
20 google-auth-oauthlib==1.2.3
21 googleapis-common-protos==1.72.0
22 greenlet==3.2.4
23 httplib2==0.31.0
24 idna==3.11
25 kombu==5.5.4
26 oauthlib==3.3.1
27 packaging==25.0
28 prompt_toolkit==3.0.52
29 proto-plus==1.26.1
30 protobuf==6.33.0
31 pyasn1==0.6.1
32 pyasn1_modules==0.4.2
33 pycparser==2.23
34 pyparsing==3.2.5
35 python-dateutil==2.9.0.post0
36 redis==7.0.1
37 requests==2.32.5
38 requests-oauthlib==2.0.0
39 requirements.py==1.0.1
40 rsa==4.9.1
41 setuptools==80.9.0
42 six==1.17.0
43 sqlparse==0.5.3
44 tzdata==2025.2
45 uritemplate==4.2.0
46 urllib3==2.5.0
47 vine==5.1.0
48 wcwidth==0.2.14
49 zope.event==6.0
50 zope.interface==8.0.1
```

Se muestra un archivo `requirements.txt` que lista las dependencias del proyecto. Incluye paquetes clave como Django para el desarrollo web, Celery para tareas asincrónicas y varias bibliotecas de Google, lo que confirma la integración del proyecto con servicios como la API de Gmail.

- Integración inicial de un microservicio o módulo independiente.



```

notify_service_app > models.py > Notification
1 from django.db import models
2 from stress.models import CustomUser
3
4 class Notification(models.Model):
5     message = models.CharField(max_length=255)
6     is_read = models.BooleanField(default=False)
7     user = models.ForeignKey(CustomUser, on_delete=models.CASCADE, related_name='notifications')
8     url = models.CharField(max_length=30)
9
10     def __str__(self):
11         return f"{self.message} | {self.user}"

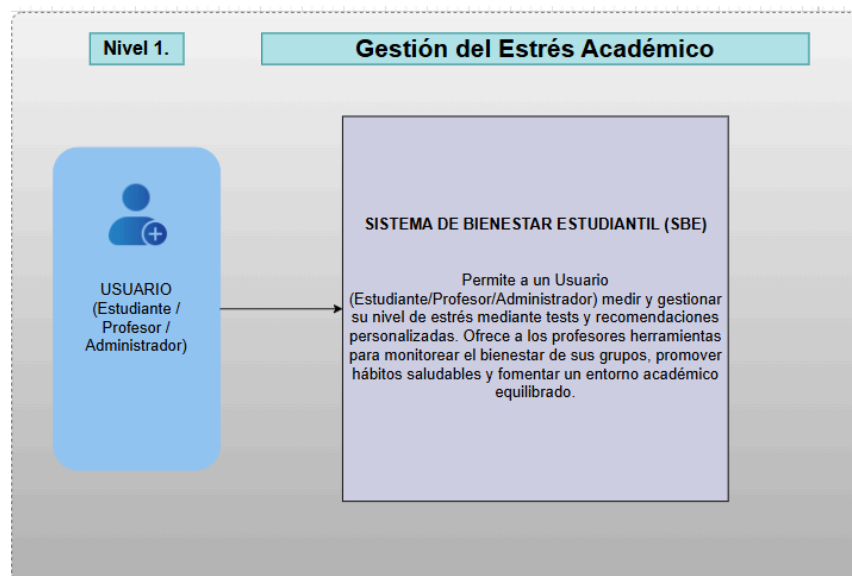
notify_service_app > views.py > mark_notification_as_read
1 from django.shortcuts import render
2 from django.shortcuts import get_object_or_404, redirect
3 from django.contrib.auth.decorators import login_required
4 from .models import Notification
5
6 # Create your views here.
7 @login_required
8 def mark_notification_as_read(request, notification_id):
9     notification = get_object_or_404(Notification, id=notification_id, user=request.user)
10
11     if notification.is_read == False:
12         notification.is_read = True
13         notification.save()
14
15     return redirect(notification.url)
16
17 @login_required
18 def notify(request, users, message, url):
19     for user in users:
20         Notification.objects.create(message=message, user=user, url=url)

```

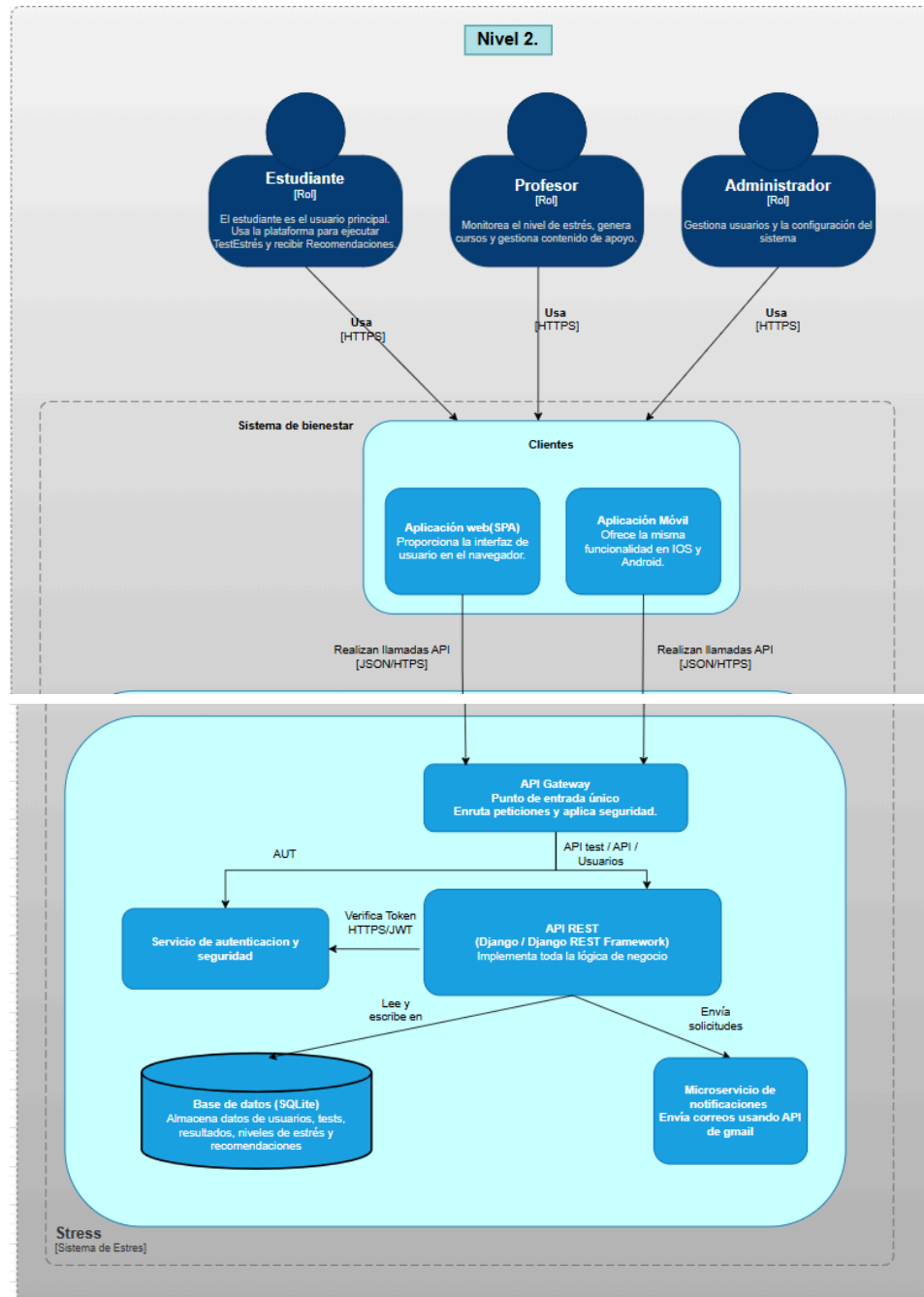
Integramos un módulo independiente llamado notify_service_app para manejar las notificaciones. Este módulo opera como un microservicio, con su propio modelo de datos (models.py) para las notificaciones y su lógica de negocio. En la clase Notification, definimos campos esenciales como el mensaje, el estado de lectura (is_read) y la relación con el usuario, manteniendo esta funcionalidad separada del resto del sistema.

2. Modelo C4 – Arquitectura del sistema:

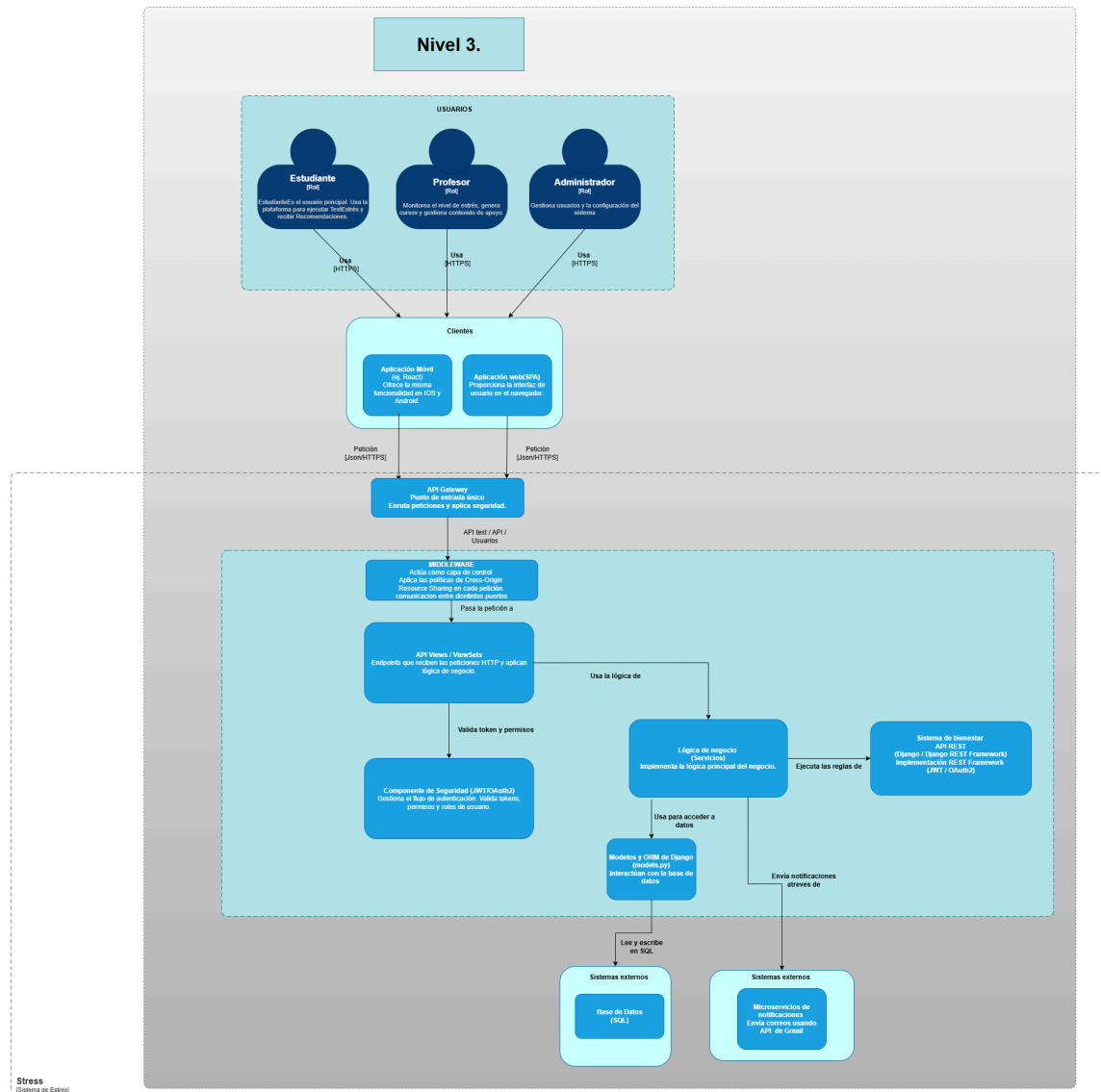
- **Nivel 1: Context Diagram** – visión general del sistema y actores externos.



- **Nivel 2: Container Diagram** – relación entre backend, frontend y base de datos.

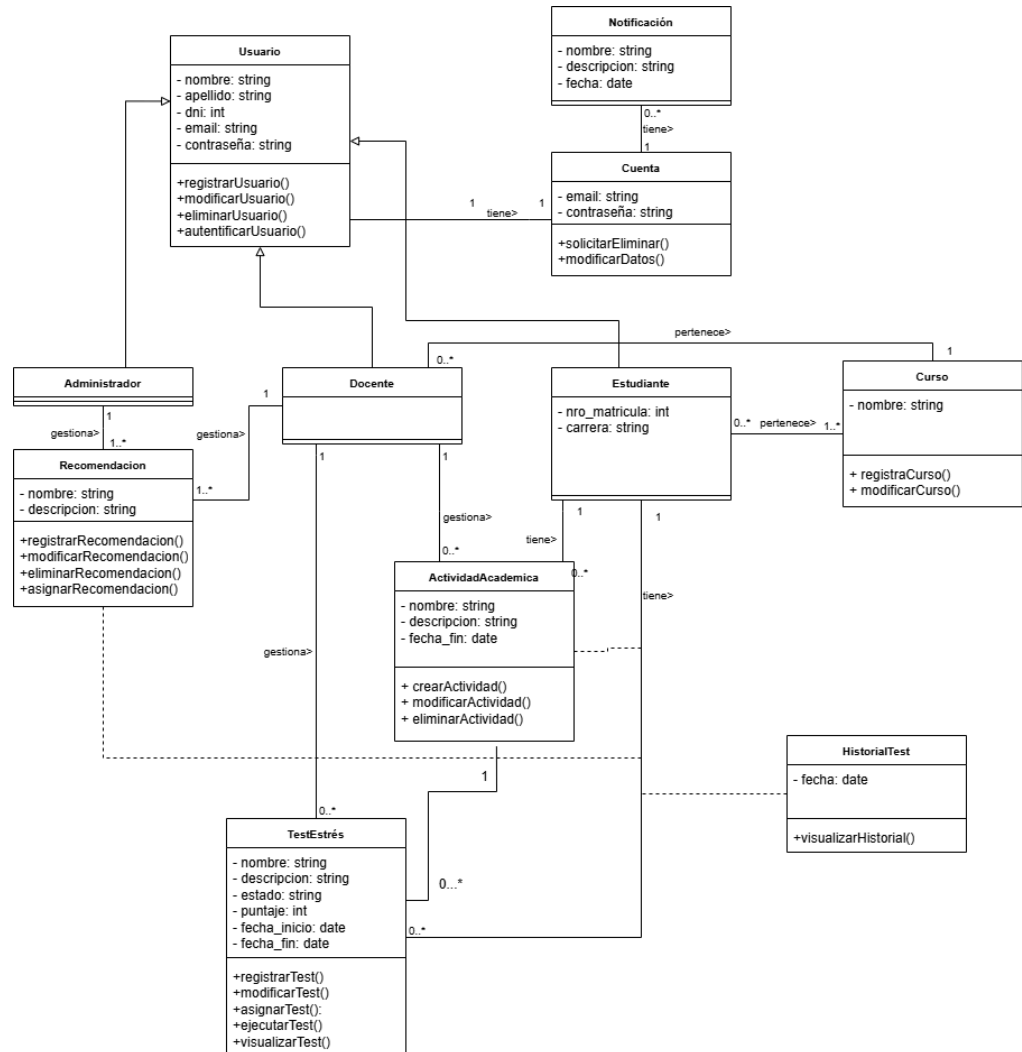


- **Nivel 3: Component Diagram** – estructura interna del backend (controladores, servicios, repositorios).

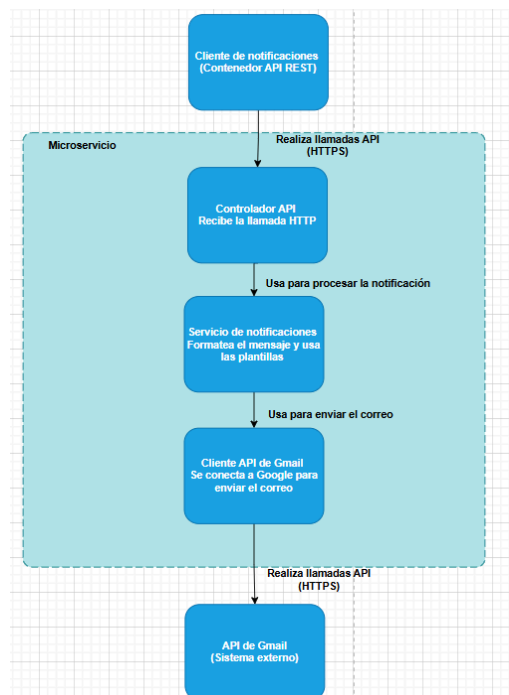


- **Nivel 4: Code Diagram** – diagramas de clase
- Diagramas elaborados en **PlantUML, Mermaid, Draw.io** o **Structurizr**, almacenados en /docs/architecture/.

Lógica del negocio se expande al modelo de clases:



Modelo del micro-servicio de notificaciones



Para mejor visualización se adjunta link de draw:

[https://app.diagrams.net/#G18klqfxADMM2DmOi8GR8tvNKrBWIXcGDh#%7B"pagelid"%3A"MAfRuSXauX6-3sCmr1Wd"%7D](https://app.diagrams.net/#G18klqfxADMM2DmOi8GR8tvNKrBWIXcGDh#%7B)

3. Documentación técnica:

- Descripción de arquitectura y dependencias.

El proyecto utiliza un conjunto de dependencias definidas en el archivo requirements.txt, las cuales permiten implementar un backend robusto, escalable y con soporte para tareas asincrónicas, autenticación segura y servicios externos.

A continuación, se describen las principales librerías y su función dentro del entorno:

```
# requirements.txt
1  amqp==5.3.1
2  asgiref==3.10.0
3  billiard==4.2.2
4  cachetools==6.2.1
5  celery==5.5.3
6  certifi==2025.10.5
7  cffi==2.0.0
8  charset-normalizer==3.4.4
9  click==8.3.0
10 click-didyoumean==0.3.1
11 click-plugins==1.1.1.2
12 click-repl==0.3.0
13 colorama==0.4.6
14 Django==5.2.7
15 event==25.9.1
16 google-api-core==2.28.1
17 google-api-python-client==2.187.0
18 google-auth==2.41.1
19 google-auth-http2==0.2.1
20 google-auth-oauthlib==1.2.3
21 googleapis-common-protos==1.72.0
22 greenlet==3.2.4
23 httplib2==0.31.0
24 idna==3.11
25 kombu==5.5.4
26 oauthlib==3.3.1
27 packaging==25.0
28 prompt_toolkit==3.0.52
29 proto-plus==1.26.1
30 protobuf==6.33.0
31 pyasn1==0.6.1
32 pyasn1_modules==0.4.2
33 pyparsing==2.2.3
34 pyparsing==3.2.5
35 python-dateutil==2.9.0.post0
36 redis==7.0.1
37 requests==2.32.5
38 requests-oauthlib==2.0.0
39 requirements.py==1.0.1
40 rsa==4.9.1
41 setuptools==80.9.0
42 six==1.17.0
43 sqlparse==0.5.3
44 tzdata==2025.2
45 uritemplate==4.2.0
46 urllib3==2.5.0
47 vine==5.1.0
48 wcwidth==0.2.14
49 zope.event==6.0
50 zope.interface==8.0.1
```

Estas dependencias permiten que nuestro sistema:

- Ejecuta procesos concurrentes y tareas programadas.
- Interactúe con servicios externos de Google y APIs REST.
- Aplique estándares de seguridad mediante OAuth2 y JWT.
- Mantenga un flujo de trabajo estable y seguro para usuarios y administradores.

- Colección Postman o documentación Swagger.

En este caso utilizamos Postman:

Se creó una colección Postman con todos los endpoints del backend, organizada por carpetas:

Auth:

```
# general
path('', views.home, name='home'),
path('login/', views.log_in, name='login'),
path('register/', views.register, name='register'),
path('profile/', views.profile, name='profile'),
path('logout/', views.log_out, name='logout'),
```

Usuarios:

```
# notification
path('notification/change/<int:notification_id>', views.mark_notification_as_read, name='notification'),
```

Notificaciones:

```
# users
path('user/state/udate/<int:user_id>', views.change_state_user, name='change-user-state'),
path('user/desactivate/', views.desactivate_account_notification, name='desactivate-account'),
```

Cada endpoint incluye ejemplos de peticiones (request body) y respuestas esperadas (status 200, 400, 401), así como el encabezado Authorization: Bearer <token> para pruebas con JWT.

- Estándares de codificación y guía de instalación.

Para garantizar la calidad, mantenibilidad y coherencia del código desarrollado, se aplicaron buenas prácticas de programación y convenciones de estilo tanto a nivel estructural como semántico.

Los principales estándares implementados fueron los siguientes:

- Estructura modular y organizada:

El proyecto se divide en módulos independientes (controladores, servicios, repositorios y rutas), lo que facilita la escalabilidad del sistema y la reutilización del código.

- Control de errores y validaciones centralizadas:

Se implementaron middlewares o decoradores para el manejo global de excepciones, validación de datos de entrada y mensajes de error coherentes.

Las respuestas del servidor siguen un formato JSON estándar con campos status, message y data.

- Buenas prácticas de seguridad:

Cifrado de contraseñas con bcrypt o librerías equivalentes.

Manejo de tokens JWT con expiración controlada.

Validación exhaustiva de entrada para evitar inyección SQL o XSS.

Cumplimiento parcial de las recomendaciones OWASP Top 10.

4. Informe de seguridad:

- Evidencia de aplicación de principios **OWASP Top 10**, autenticación JWT u OAuth2 y cifrado de contraseñas.

```
class PasswordInput(Input):
    input_type = "password"
    template_name = "django/forms/widgets/password.html"

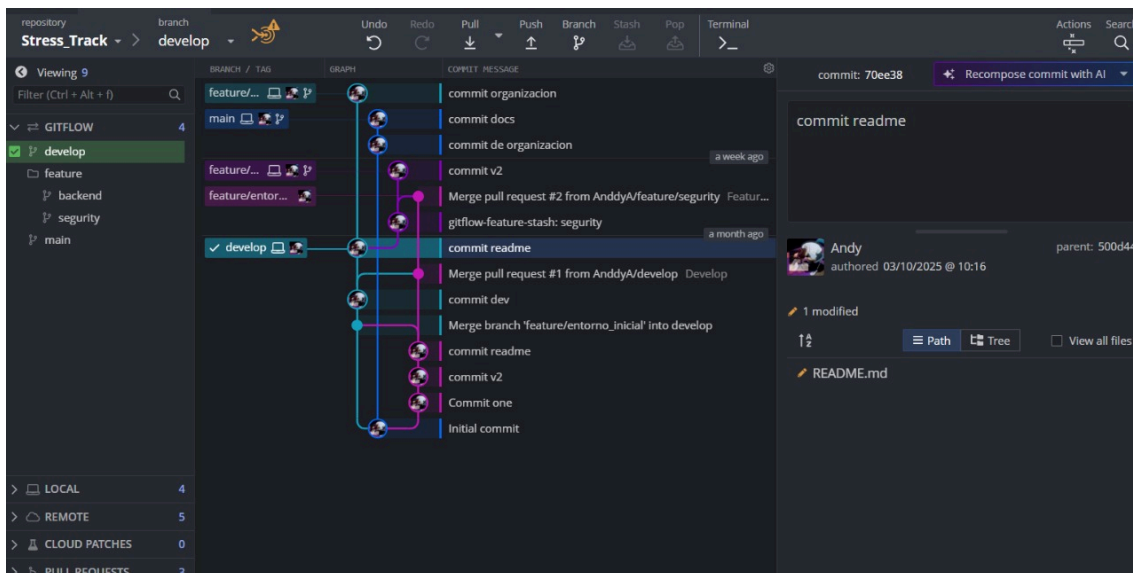
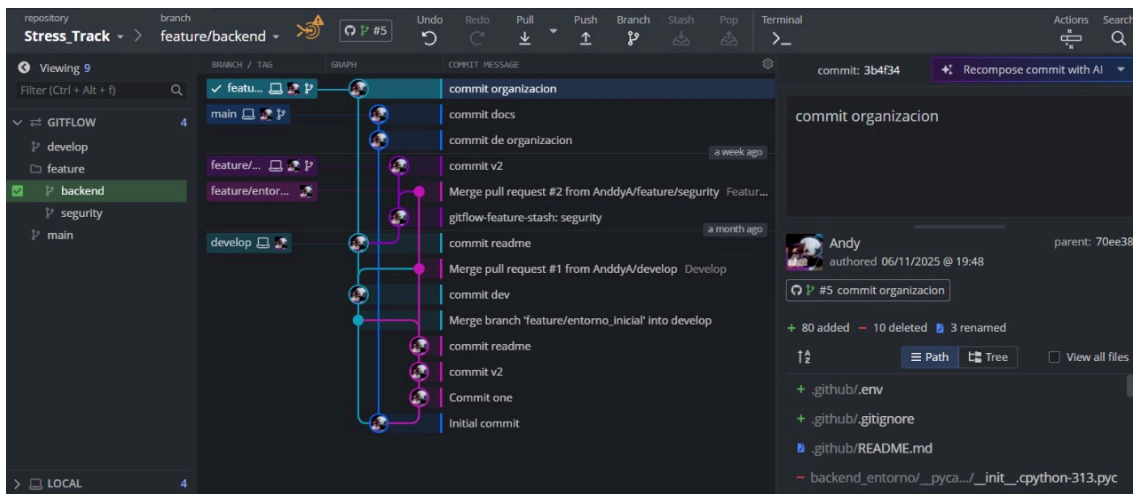
    def __init__(self, attrs=None, render_value=False):
        super().__init__(attrs)
        self.render_value = render_value

    def get_context(self, name, value, attrs):
        if not self.render_value:
            value = None
        return super().get_context(name, value, attrs)
```

	first_name	email	password
	Filter...	Filter...	Filter...
1	Administrador	admin@unl.edu.ec	pbkdf2_sha256\$1000000\$u42rlw7OtyoPdb1WyDkHGf\$...
2	Andy	andy.apolo@unl.edu.ec	pbkdf2_sha256\$1000000\$jFNQ4IX8UfjxL8cDhK3kv\$soY2...
3	Cristian	doc@unl.edu.ec	pbkdf2_sha256\$870000\$soVIGro457xhX6nGdWaCSU3\$2...
4	Marylin	marylin.alvarado@unl.edu.ec	pbkdf2_sha256\$870000\$3HCLyFlrn5sTKfyabchTfb\$81jpp...
5	Andersson	doc2@unl.edu.ec	pbkdf2_sha256\$870000\$p74X8R6wKwWDDzyp9fUK0\$...
6	Andy	jahir@unl.edu.ec	pbkdf2_sha256\$870000\$69QMdCtj38zSQLjjVeUdA4\$ueq...
7	Kevin	kkevin@unl.edu.ec	pbkdf2_sha256\$870000\$yKu8ojqnmG1bqwUU16MXKm...
8	Jairo	jairo@ajilaunl.edu.ec	pbkdf2_sha256\$870000\$tufj1pR9OzfY632byR2aHc\$OD...
9	Marco	marco@unl.edu.ec	pbkdf2_sha256\$870000\$ggevoUDsTbrswsgdViomLD\$Qj...
10	Elias	elias.poma@unl.edu.ec	pbkdf2_sha256\$870000\$es21fDayuhqu8CI9Bb9KzV\$nG...
11	Justin	justin.gutierrez@unl.edu.ec	pbkdf2_sha256\$870000\$gCNVyyUCIK3mmQ1ufahXCI\$...

5. Control de versiones con GitKraken y flujo GitFlow:

- Ramas principales: main, develop, feature/*.
- Commits descriptivos y merges ordenados.



Herramientas y tecnologías sugeridas:

- **Frameworks backend:** Django REST Framework, Flask, Express.js o Spring Boot.
- **Modelado C4:** PlantUML, Mermaid, Structurizr o Draw.io, LucidChart.
- **Control de versiones:** Git y GitKraken (flujo GitFlow).
- **Pruebas:** Postman o Swagger.
- **Seguridad:** JWT, bcrypt, validaciones OWASP.

Forma de entrega en el EVA:

- **Texto en línea:** URL del repositorio remoto (GitHub/GitLab).
- **Archivo adjunto:** PDF con documentación técnica, diagramas C4 exportados y capturas de pruebas de la API.

2. Conclusiones

- La aplicación del modelo C4 fue fundamental para establecer una arquitectura base robusta y comprensible para el sistema multiplataforma. Este enfoque de diseño nos permitió visualizar y definir claramente las fronteras del sistema, las interacciones entre los actores, los frontends, el backend y la base de datos, así como la estructura interna de componentes.
- La implementación de estándares de seguridad desde el inicio del backend ha sido un pilar clave del desarrollo. Al integrar la autenticación basada en tokens JWT, el cifrado de contraseñas y aplicar las validaciones de entrada sugeridas por OWASP, se ha construido un servicio REST que protege la información sensible de los usuarios. La creación de una estructura modular (separando controladores, servicios y lógica de negocio) y el manejo centralizado de errores refuerzan la fiabilidad y mantenibilidad del sistema ante futuras ampliaciones o posibles ataques.
- La adopción de un flujo de trabajo profesional, combinando el control de versiones GitFlow con la validación de endpoints mediante Postman, demostró ser vital para la eficiencia del equipo y la calidad del producto. El uso de ramas permitió un desarrollo ordenado y aislado de las funcionalidades, mientras que la colección de Postman sirvió como una forma de documentación ejecutable.

3. Recomendaciones

- Se recomienda establecer una política de documentación como código, para mantener los diagramas del modelo C4 sincronizados con la evolución del código. Esto evitará que los diagramas se vuelvan obsoletos a medida que se añadan los nuevos contenedores del frontend y la app móvil en las siguientes unidades.
- Si bien se implementaron las bases de seguridad con JWT, se sugiere para una aplicación real profundizar en la implementación de más principios OWASP. Esto incluiría añadir mecanismos de rate limiting (límite de peticiones) en los endpoints de autenticación para prevenir ataques de fuerza bruta y establecer una política de permisos (roles) más granular. Esto último será crucial para diferenciar las acciones que puede realizar un administrador frente a un usuario estándar dentro del sistema.
- Se recomienda evolucionar el sistema de un modelo de monitoreo pasivo a uno de intervención proactiva. En futuras versiones, el sistema podría ser capaz de analizar los datos de estrés de los estudiantes (provenientes de los tests) de forma anónima y agregada por curso o paralelo. Si la plataforma detecta un pico de estrés colectivo en un grupo específico, podría generar una alerta automática (no punitiva) al profesor responsable, sugiriendo que la carga académica en ese momento es crítica, permitiéndole tomar acciones pedagógicas como ajustar plazos o proveer material de apoyo.

4. Evaluación

Rúbrica de evaluación (escala 2–1–0)

Criterio	2 – Logro Alto	1 – Logro Medio	0 – Logro Bajo / No evidencia
1. Diseño e implementación del backend	Backend funcional, modular y completo con endpoints correctos.	Backend funcional con errores menores o estructura incompleta.	No presenta un backend funcional o está desorganizado.
2. Aplicación del modelo C4	Presenta correctamente los niveles Context, Container y Component coherentes con el diseño.	Diagramas incompletos o coherencia entre niveles.	No presenta el modelo C4 o es incorrecto.
3. Seguridad y validaciones	Aplica JWT/OAuth2, cifrado y medidas de seguridad OWASP adecuadamente.	Implementa seguridad parcial con fallos menores.	Sin autenticación ni medidas de seguridad.
4. Control de versiones (GitFlow)	Aplica correctamente GitFlow (main, develop, feature/*) con commits descriptivos.	Uso básico o incompleto del flujo control de versiones.	No se evidencia estructura incorrecta.
5. Documentación y estructura del repositorio	Repositorio ordenado, estructura conforme al estándar y README completo.	Estructura parcial o documentación incompleta.	Sin estructura ni documentación técnica.

Articulación con el proyecto global:

- El backend y los diagramas C4 constituyen la base del sistema.
- En la Unidad 2, se ampliará el Container y Component Diagram para incluir el frontend.
- En la Unidad 3, se integrará el Code Diagram (nivel 4 del C4) con la aplicación móvil.
- Todos los artefactos formarán parte del portafolio digital final y la entrega sumativa del proyecto completo.